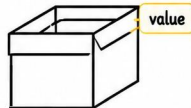




1. What is a Variable?

A variable stores a value so you can use it later in your program.



2. Why Variables Matter

- They help store data.
- They let you reuse values.
- They allow you to update information.
- They make code easier to read.

3. let

`let`

Use `let` when the value may change later.

```
let age = 20;
```

4. const



Use `const` when the value should not be reassigned.

```
const country =  
"Iran";
```

5. var

`var`

An older way to declare variables. Prefer `let` and `const` in modern code.

```
var name = "Ali";
```

6. Choosing let or const



- Use `const` by default.
- Use `let` only when the value must change.

7. Variable naming rules



- Names can use letters, numbers, `_` and `$`.
- They cannot start with a number.
- They should not use reserved words.



8. camelCase

Aa

Common JavaScript style for variable names.

```
firstName  
totalPrice  
userAge
```

9. What is a Data Type?

?

A data type describes the kind of value stored in a variable.

abc

123

true

[]

10. String

abc

Text inside quotes.

Example:

```
const name = "Ayla";
```

You can use single
or double quotes.

‘ ’ “ ”

11. Number

123

Whole numbers or decimals.

Examples:

```
20  
24.5  
-5
```

No quotes
around numbers.

12. Boolean

Only two values: true or false.

Example:

```
let isStudent = true;
```

true

Remember:
true has no quotes!



13. Undefined

undefined

A variable exists, but no value has been assigned yet.

Example:

```
let favoriteColor;
```

The variable has no value yet.

14. Null

null

null means an intentionally empty value.

Example:

```
let selectedUser = null;
```

We set it to null on purpose.

15. Undefined vs null

**undefined**

Variable declared but no value assigned yet.

null

Variable has an intentional empty value.

16. typeof

typeof

Use typeof to check the type of a value or variable.

Example:

```
typeof "Hello";
```

Returns the type: "string"





1. Undefined

`undefined`

A variable exists, but no value has been assigned yet.

Example:

```
let favoriteColor;
```



The variable has no value yet.



2. Null

`null`

null means an intentionally empty value.

Example:

```
let selectedUser = null;
```



We set it to null on purpose.



3. Undefined vs Null

`undefined`

Variable declared but no value assigned yet.

`null`

Variable has an intentional empty value.



4. typeof

`typeof`

Use typeof to check the type of a value or variable.

Example:

```
typeof "Hello";
```



Returns the type: "string"





1. Type Examples

Check the type of different values.

- `typeof "Hello"` → `"string"`
- `typeof 25` → `"number"`
- `typeof true` → `"boolean"`



Use `typeof` to identify data types.



2. Changing a Value

Variables declared with `let` can change.

Example:

```
let score = 10;  
  
score = 15;
```



The value is updated to 15.



3. Quick Example

Different declarations and their types.

```
const firstName = "Ayla";  
let age = 20;  
const isStudent = true;  
let job;
```

- `firstName` → `string`
- `age` → `number`
- `isStudent` → `boolean`
- `job` → `undefined`



4. Common Mistakes

Avoid these mistakes:

- Redeclaring a `let` variable.
- Trying to reassign a `const` variable.
- Forgetting quotes for strings.
- Using invalid variable names (e.g., `2name`).



Small mistakes can cause big problems!





1. Practice

Try these steps yourself:

- ✓ Create variables:
`firstName`, `age`, `learningJavaScript`,
`favoriteColor`.
- ✓ Print them using `console.log()`.
- ✓ Use `typeof` to check their types.
- ✓ Change the value of `age`.

// Example

```
let firstName = "Ayla";  
let age = 20;  
console.log(typeof firstName);  
console.log(typeof age);
```



2. Challenge Answer

If the variables are set like this:

```
const firstName = "Ayla";  
let age = 20;  
const isStudent = true;  
let job;
```

Then their types are:

- `firstName` → `"string"`
- `age` → `"number"`
- `isStudent` → `"boolean"`
- `job` → `"undefined"`



3. Mini Project

Create a simple Student Profile using variables.

Example:

```
const firstName = "Ayla";  
const course = "JavaScript";  
let currentDay = 2;  
let isLearning = true;  
let completedLessons = 1;
```



Use `console.log()` to print each value.



4. Key Takeaway

Remember these key points:

- Variables store data.
- `const` = no reassignment.
- `let` = value can change.
- Main types:

string

number

boolean

undefined

null

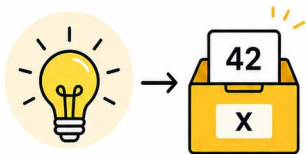


`typeof` helps identify types.



خلاصه درس ۲: متغیرها و انواع داده در جاوا اسکریپت

۱ متغیر چیست؟



متغیر (Variable) محلی برای نگهداری یک مقدار در برنامه است. با کمک متغیرها می‌توانیم داده‌ها را ذخیره کنیم، بعداً دوباره از آن‌ها استفاده کنیم و مقدارشان را در صورت نیاز تغییر دهیم.

۲ چرا به متغیر نیاز داریم؟



ذخیره‌سازی داده‌ها



استفاده دوباره از مقادیر



به‌روزرسانی اطلاعات در طول برنامه



خوانا تر شدن کد

متغیرها باعث می‌شوند کد منظم‌تر، قابل‌فهم‌تر و کاربردی‌تر شود.



خلاصه درس ۲: { } متغیرها و انواع داده در جاوا اسکریپت



۳ معرفی let ، const و var

let

برای متغیرهایی که ممکن است مقدارشان بعداً تغییر کند.

const

برای مقادیری که قرار نیست دوباره مقداردهی شوند.

var

روش قدیمی‌تر تعریف متغیر در جاوا اسکریپت که امروز کمتر استفاده می‌شود.



۴ قاعده سریع انتخاب

✓ به طور پیش‌فرض از **const** استفاده کن.

✓ اگر مقدار قرار است تغییر کند، از **let** استفاده کن.

✓ فعلاً **var** را فقط بشناس؛ در کدهای جدید معمولاً انتخاب اول نیست.

یادگیری درست متغیرها، پایه‌ای محکم برای درک بهتر جاوا اسکریپت است.





1 قوانین نام گذاری متغیرها

- نام متغیر می تواند از حروف، عدد، _ و \$ ساخته شود.
- نام متغیر نباید با عدد شروع شود.
- در نام متغیر نباید فاصله وجود داشته باشد.
- نباید از کلمات رزرو شده زبان استفاده کنیم.



2 camelCase چیست؟

در جاوا اسکریپت معمولاً نام متغیرها را با روش camelCase می نویسیم؛ یعنی کلمه اول با حرف کوچک شروع می شود و ابتدای کلمه های بعدی با حرف بزرگ نوشته می شود.

firstName

userAge

totalPrice

isLearningJavaScript



خلاصه درس ۲

متغیرها و انواع داده در جاوااسکریپت

Lesson 02 Summary

نام‌های درست و نادرست

۳



درست

- userName
- favoriteColor
- coursePrice



نادرست

- 2name
- user name
- first-name
- let



انتخاب نام مناسب باعث می‌شود کد سریع‌تر فهمیده شود.

چند مثال از تعریف متغیر

۴



```
const city = "Tehran";  
let userAge = 20;  
const isStudent = true;  
var oldName = "Ali";
```



در کدهای امروزی معمولاً `const` و `let` را به `var` ترجیح می‌دهیم.



خلاصه درس ۲: متغیرها و انواع داده در جاوااسکریپت

Lesson 02 Summary

JS

1 نوع داده چیست؟



نوع داده (Data Type) مشخص می‌کند مقداری که داخل متغیر ذخیره شده از چه جنسی است؛ مثلاً متن، عدد یا مقدار منطقی.

2 رشته یا String



رشته برای نگهداری متن استفاده می‌شود و باید داخل کوتیشن نوشته شود.

```
const name = "Ayla";  
const course = 'JavaScript';
```

برای رشته می‌توان از کوتیشن تکی یا دوتایی استفاده کرد.



JS

خلاصه درس ۲: متغیرها و انواع داده در جاوااسکریپت

Lesson 02 Summary

123

عدد یا Number 3

نوع Number برای اعداد صحیح و اعشاری استفاده می‌شود.

20

24.5

-5



اعداد بدون کوتیشن نوشته می‌شوند.



بولی یا Boolean 4

نوع Boolean فقط دو مقدار دارد: true و false. از این نوع معمولاً برای وضعیت‌های درست/نادرست استفاده می‌کنیم.

```
let isStudent = true;  
const isLoggedIn = false;
```



true و false نباید داخل کوتیشن باشند.





خلاصه درس ۲:

JS

متغیرها و انواع داده در جاوااسکریپت

Lesson 02 Summary



1 Undefined چیست؟

وقتی یک متغیر تعریف می‌شود اما هنوز مقداری برای آن قرار نداده‌ایم، مقدار آن undefined است.

```
let favoriteColor;
```



یعنی متغیر وجود دارد اما هنوز مقدار نگرفته است.



2 چیست Null؟

null یعنی یک مقدار خالی که عمداً خودمان آن را تعیین کرده‌ایم.

```
let selectedUser = null;
```



در null خودمان تصمیم می‌گیریم مقدار خالی باشد.



{ }

خلاصه درس ۲:

متغیرها و انواع داده در جاوااسکریپت

Lesson 02 Summary

{ }

JS

3 تفاوت null و undefined



null

متغیر یک مقدار
خالی عمدی دارد.

undefined

متغیر تعریف شده ولی
هنوز مقدار ندارد.



این تفاوت کوچک، در درک کد بسیار مهم است.

4 typeof چیست؟

با typeof می‌توانیم نوع یک مقدار یا متغیر را بررسی کنیم.

typeof "Hello";	→	"string"
typeof 25;	→	"number"
typeof true;	→	"boolean"





1 تغییر مقدار متغیر

متغیرهایی که با `let` تعریف می‌شوند می‌توانند در ادامه برنامه مقدار جدید بگیرند.

```
let score = 10;  
score = 15;
```



در این مثال مقدار `score` از 10 به 15 تغییر کرده است.



2 یک مثال کامل

```
const firstName = "Ayla";  
let age = 20;  
const isStudent = true;  
let job;
```

- `firstName` → string
- `age` → number
- `isStudent` → boolean
- `job` → undefined



در `null` خودمان تصمیم می‌گیریم مقدار خالی باشد.

{ }

خلاصه درس ۲:

متغیرها و انواع داده در جاوااسکریپت

Lesson 02 Summary

{ }



JS

3 خطاهای رایج



- مقداردهی دوباره به `const`
- فراموش کردن کوتیشن برای رشته‌ها
- استفاده از نام نامعتبر برای متغیر
- اشتباه گرفتن `undefined` با `null`

4 تا اینجا باید چه چیزهایی بلد باشیم؟



- ✓ متغیر چیست و چه کاری انجام می‌دهد.
- ✓ تفاوت `const` ، `let` و `var` .
- ✓ مفهوم نوع داده و انواع اصلی آن.
- ✓ مفهوم `undefined` ، `null` و `typeof` .



1 تمرین پیشنهادی

- چهار متغیر با نام‌های `firstName` ، `age` ، `favoriteColor` و `isLearning` تعریف کن.
- مقدار مناسب برای هرکدام قرار بده.
- با `console.log()` آن‌ها را چاپ کن.
- با `typeof` نوع هرکدام را بررسی کن.
- مقدار `age` را تغییر بده و دوباره چاپ کن.



2 جواب Challenge

```
const firstName = "Ayla";  
let age = 20;  
const isStudent = true;  
let job;
```

- نوع `firstName` برابر `string` است.
- نوع `age` برابر `number` است.
- نوع `isStudent` برابر `boolean` است.
- نوع `job` برابر `undefined` است.

JS

خلاصه درس ۲ — متغیرها و انواع داده در جاوا اسکریپت

Lesson 02 Summary



3 مینی پروژه

یک Student Profile ساده بساز.

```
const firstName = "Ayla";  
const course = "JavaScript";  
let currentDay = 2;  
let isLearning = true;  
let completedLessons = 1;
```

می‌توانی با `console.log()` اطلاعات این پروفایل را نمایش بدهی.



4 جمع‌بندی نهایی

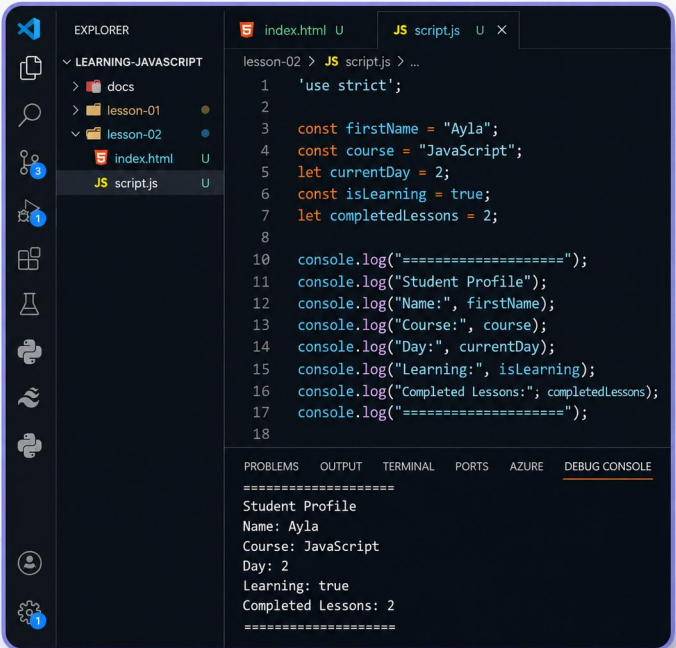
- متغیرها برای ذخیره و مدیریت داده‌ها هستند.
- `const` برای مقادیر ثابت و `let` برای مقادیر قابل تغییر مناسب است.
- انواع مهم امروز: `string` ، `number` ، `boolean` ، `undefined` ، `null` .
- `typeof` برای تشخیص نوع داده به ما کمک می‌کند.

پایه‌های مهم جاوا اسکریپت را ساختی؛ حالا آماده‌ای قدم بعدی را برداری.



خروجی تمرین جاوااسکریپت

JavaScript Practice Output



The screenshot shows a code editor with a dark theme. On the left is the Explorer sidebar showing a file structure: 'LEARNING-JAVASCRIPT' containing 'docs', 'lesson-01', and 'lesson-02'. Inside 'lesson-02' are 'index.html' and 'JS script.js'. The main editor area has two tabs: 'index.html' and 'JS script.js'. The 'JS script.js' tab is active, showing the following code:

```
1 'use strict';
2
3 const firstName = "Ayla";
4 const course = "JavaScript";
5 let currentDay = 2;
6 const isLearning = true;
7 let completedLessons = 2;
8
9
10 console.log("=====");
11 console.log("Student Profile");
12 console.log("Name:", firstName);
13 console.log("Course:", course);
14 console.log("Day:", currentDay);
15 console.log("Learning:", isLearning);
16 console.log("Completed Lessons:", completedLessons);
17 console.log("=====");
18
```

At the bottom of the editor is the 'DEBUG CONSOLE' tab, which displays the output of the code:

```
=====
Student Profile
Name: Ayla
Course: JavaScript
Day: 2
Learning: true
Completed Lessons: 2
=====
```